LECTURE SCRIPT

DQN

1 Training Neural Networks to imitate the Q-function

A neural network represents a function f_θ whose weights and biases are collected in a parameter vector θ. We cannot generally just set the parameters so that the network represents a specified function: its outputs depend jointly and nonlinearly on many shared parameters, and the chosen architecture may only approximate that function. Instead, we train on examples (x_i, y_i) with $y_i = f(x_i)$, or on noisy observations of these values. A *loss function* measures how far a prediction $f_\theta(x_i)$ is from its target y_i. A common choice is the quadratic loss, possibly averaged over a batch B:

$$L_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \big(f_\theta(x_i) - y_i\big)^2. \tag{1}$$

Squaring treats positive and negative errors alike, penalises large errors more strongly, and gives a smooth loss with a simple derivative. For noisy targets, minimising expected squared loss recovers the conditional mean. Quadratic loss is useful, but its sensitivity to large errors can also be a disadvantage. Backpropagation then computes the gradient of the loss and an optimiser uses it to adjust the parameters. For example, stochastic gradient descent (SGD) uses $\theta \leftarrow \theta - \eta \nabla_\theta L_B(\theta)$ with learning rate $\eta > 0$. Modern optimisers such as Adam adapt these steps using gradient history. Each step changes the function represented by the network into the direction of the minimum, where the network is as close to the target as possible.

Q-learning with a Neural Network

A Q-network, approximating the Q-function may now take a state as input and outputs a vector of Q-values, one entry per possible action, i.e., $Q(s, \cdot\,; \theta)$ produces the set $\{Q(s, a; \theta)\}_{a \in \mathcal{A}}$. Thinking about how to train this network, we can take two perspectives, for now ignoring the fact that the functions involved are neural networks and work in the tabular case:

1. **Machine-learning view:** we want to fit a target to the data. The Q-function has the property that it is the fixed point of the Bellman optimality operator, so at time n we want our function $Q_n(s, a)$ to fit the target $y_n = r + \gamma \max_{a'} Q_n(s', a')$ as closely as possible for any transitions (s, a, r, s') we observe:

$$\begin{aligned} \widetilde{L}_n^Q &= \mathbb{E}[(\mathbb{E}[y_n] - Q_n(s, a))^2] \\ &= \mathbb{E}[(y_n - Q_n(s, a))^2] + \mathbb{E}[\mathrm{V}(y_n)] = L_n^Q + \mathbb{E}[\mathrm{V}(y_n)]. \end{aligned} \tag{2}$$

   At this point it may not be clear why we want to only minimize L_n^Q in the following. However, in the context of DQN it will make sense later.

2. **Statistical/RL view:** we want to approximate the Bellman optimality operator because it is a contraction in the sup-norm, so repeated application converges to the optimal Q. A well-known statistical trick reveals that $\mathbb{E}[X] = \arg\min_\theta \mathbb{E}[(X - \theta)^2]$. Thus, we have

$$T^*Q(s, a) = \mathbb{E}_{r, s'}\left[r + \gamma \max_{a'} Q(s', a')\right] = \arg\min_\theta \mathbb{E}[(y_n - \theta)^2]. \tag{3}$$

Doing one-step SGD with stepsize $\frac{\alpha}{2}$ on L_n^Q or on the minimization problem from the second view gives exactly the same thing, namely, the Q-Learning update:

$$Q_{n+1}(s, a) \leftarrow Q_n(s, a) + \alpha\big(y - Q_n(s, a)\big). \tag{4}$$

In the tabular case we can thus extend Q-Learning to a function approximation problem. The tricks we will see in the next section that characterize DQN and are necessary to make it work, are usually motivated by the first view, but we think the second view is much more natural, specifically for one of the tricks.

2 Deep Q-Network (DQN)

Introduction – Deep Q-Network (DQN)

Deep Q-Network (DQN), as proposed by Mnih et al. (DeepMind 2015), is a Q-learning algorithm that uses a deep neural network to approximate the action-value function. They use the ALE (Arcade learning environment), then consisting of 49 games (now: 57 games) to show that DQN can match human level performance.

One of the challenges that DQN addresses is how to deal with a large, continuous state–action space like that of Atari games: high-dimensional pixel observations must be processed before Q-learning can be applied. The first component are convolutional neural networks (CNNs), which process these images before fully connected layers produce one Q-value per possible action.

Problems of Q-learning with Neural Networks

Before DQN, there were two major probelems with naive function approximation by NNs that led to instability and divergence in Q-learning:

- NNs can forget
- Even small Q-updates can massively influence the data distribution
- Correlation between successive samples and between the action chosen and the target values.

DQN Architecture

The architecture used in the original DQN ("Nature CNN") is as follows:

- **Pre-processing:** the environment is rendered at 84×84 pixels and converted to grayscale. Four consecutive frames are stacked to capture motion. These pixel values together with the score are the input to the neural network.
- **Convolutional layers:** three convolutional layers are used. The first layer has 32 filters of size 8×8 with stride 4, the second has 64 filters of size 4×4 with stride 2 and the third has 64 filters of size 3×3 with stride 1. Each convolution is followed by a ReLU activation.
- **Fully connected layer:** the flattened convolutional output is passed through a fully connected layer with 512 units and ReLU activation.
- **Output layer:** one linear output for each admissible action (e.g., 4–18 actions on Atari). Thus the network outputs the vector $Q(s, \cdot; \theta)$ for the given state.

Layer	Description	Dimensions
Input	Grayscale frames (stack of 4)	$4 \times 84 \times 84$
Conv1	8×8 kernel, stride 4, ReLU	$32 \times 20 \times 20$
Conv2	4×4 kernel, stride 2, ReLU	$64 \times 9 \times 9$
Conv3	3×3 kernel, stride 1, ReLU	$64 \times 7 \times 7$
Fully Connected	Flatten and dense layer, ReLU	512
Output	Linear layer for actions	$\lvert\mathcal{A}\rvert$

Table 1: DQN architecture showing layer descriptions and dimensions.

Experience Replay

Experience replay addresses the correlation problem between the generated data and the policy and the fact that training neural networks due to them forgetting requires batch sampling, which is not possible in the original Q-Learning algorithm by maintaining a replay buffer D of past transitions. The algorithm fills the buffer gradually while the agent acts (e.g. using an ε-greedy policy or any other behavior policy). During training, mini-batches of transitions are sampled uniformly at random from the buffer. Batch updates are performed with sampled transitions, and if the buffer is full, the oldest transitions are discarded. Random sampling and re-use of past experience both reduce variance and improve data efficiency.

Target Networks

To reduce oscillations and divergence, as well as break the correlation between the target data distribution and the network to be updated, DQN uses a target network with parameters θ^-. The target network is updated to the current network's parameters only every N steps or via a soft update $\theta^- \leftarrow \tau\theta^- + (1-\tau)\theta$ with τ close to 1 (although there is no theoretical reason to do so). Between updates, θ^- remains fixed, making the target $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ independent of the current network's rapid changes and thereby stabilising training. The above motivation is grounded in the first view of Q-Learning as fitting your network to a moving target (that is now made stationary). We can directly see why we only considered L_n^Q rather than $\widetilde{L}_n^Q$. The reason is, that now the variance term is no longer dependend on the parameter to update:

$$\widetilde{L}_n(\theta) = \mathbb{E}[(\mathbb{E}[y_n(\theta^-)] - Q_n(s,a;\theta))^2] = L_n(\theta) + \mathbb{E}[\mathrm{V}(y_n(\theta^-))]. \tag{5}$$

The second view offers a much simpler insight. Fixing the target network for N steps simply corresponds to applying N-step SGD to the minimization problem

$$\begin{aligned} T^*Q(s,a;\theta^-) &= \mathbb{E}_{r,s'}\left[r + \gamma \max_{a'} Q(s', a'; \theta^-)\right] \\ &= \arg\min_{\theta} \mathbb{E}[(y_n(\theta^-) - \theta)^2]. \end{aligned} \tag{6}$$

Maintaining a second network doubles memory. Many modern approaches thus argue to employ other stabilisation techniques, which makes sense if the only reason to employ target networks is stability. In cases like RL for large modern models such as LLMs, keeping target networks is prohibitively expensive and should probably not be done. However, given the second view, we argue that if one can afford to keep them, one should do so.

There is a slight technicality due to the application of neural networks that makes the update rule of DQN really differ from tabular Q-Learning. The reason is that we take the gradient with respect to the weights θ:

$$\nabla_\theta l_n(\theta) = \frac{1}{2}(y_n(\theta^-) - Q_n(s,a;\theta))\nabla_\theta Q_n(s,a;\theta) \tag{7}$$

leading to

$$Q_{n+1}(s,a;\theta) = Q_n(s,a;\theta) + \alpha((y_n(\theta^-) - Q_n(s,a;\theta))\nabla_\theta Q_n(s,a;\theta). \tag{8}$$

Algorithm 1: DQN

Initialize replay buffer D, Q-network parameters θ, and target network $\theta^- \leftarrow \theta$;
  choose buffer capacity $|D|$, batch size $|B|$, step size α, and target update interval N.
repeat
  Collect and pre-process a transition (s, a, s', r) using a behavior policy and add it to D.
  Sample a mini-batch B of $|B|$ transitions uniformly from D.
  Compute targets $y_i = r_i + \gamma \max_{a'} Q_n(s_i', a'; \theta^-)$.
  Take an optimiser step using $\nabla_\theta l_n(\theta) = \frac{1}{2|B|}\sum_{i\in B}(y_i - Q_n(s_i, a_i; \theta))\nabla_\theta Q_n(s_i, a_i; \theta)$.
  Every N steps, update the target network: $\theta^- \leftarrow \theta$.
until the training budget is exhausted.

3 Towards Modern Deep RL

Several algorithmic improvements to DQN, some of which you have already seen in the tabular case, include:

- **Double Q-learning / Double DQN:** Q-learning tends to over-estimate action values because the same network is used both to select and evaluate the max operator. Double Q-learning maintains two independent networks Q^A and Q^B. One network selects the greedy action and the other evaluates it:

$$\begin{aligned} Q^A(s,a) &= Q^A(s,a) + \alpha\big[r + \gamma Q^B(s', \arg\max_{a'} Q^A(s',a')) - Q^A(s,a)\big] \\ &= Q^A(s,a) + \alpha\big[r + \gamma Q^A(s', \arg\max_{a'} Q^A(s',a')) - Q^A(s,a)\big] \\ &\quad + \gamma\big(Q^B(s', \arg\max_{a'} Q^A(s',a')) - Q^A(s', \arg\max_{a'} Q^A(s',a'))\big) \end{aligned} \tag{9}$$

  To avoid managing four networks in DQN, Double DQN uses the current network to select the best action and the target network to evaluate it, thus replacing the full copy by the target network.

- **Clipped double Q / TD3:** Another approach clips the bias-term (the latter one in Double Q-Learning) to zero if it is positive, which further reduces over-estimation (it explicitly adds the same negative bias but clips it to zero if it were to be positive). The deep algorithm (including some other tricks) is called Twin Delayed Double Deep Q Networks.

- **Rainbow DQN:** Combining multiple improvements: Double DQN, dueling architecture, prioritized replay, noisy nets, multi-step returns and distributional RL. There are many further such tricks that can be combined.

Continuous Action Spaces

DQN's output layer has one entry for each action. For a continuous action space, such as a real-valued steering angle or a vector of motor torques, we cannot allocate an output unit to every possible action. A natural adaptation is a network that takes both the state and an action as input and outputs a single value:

$$(s,a) \longmapsto Q(s,a;\theta) \in \mathbb{R}. \tag{10}$$

> **Why can we not simply do the same as in DQN now?**
>
> Think about what the problem with a DQN-style algorithm would be if we had a continuous action space. Solving this problem will lead us to the algorithm Deep Deterministic Policy Gradient (DDPG) and its further developments like Soft Actor-Critic (SAC).

Loss Functions and Normalization

While the original DQN minimises the squared TD error, the paper mentions as a remark that clipping the error or using the so-called Huber loss often works better. The TD error $\delta = r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta)$ can be clipped into the range $[-1,1]$ before computing the loss: the loss is quadratic around zero and linear for large errors. The Huber loss is defined as

$$L_\tau(a) = \begin{cases} \frac{1}{2}a^2, & \text{if } |a| \le \tau, \\ \tau(|a| - \frac{\tau}{2}), & \text{otherwise,} \end{cases} \tag{11}$$

where τ is a threshold hyperparameter and is typically chosen as 1. This loss becomes the squared loss near zero and the absolute loss for larger errors. Using this loss helps to prevent very large gradients when the TD error is large and intuitively helps with stability because large updates are avoided.

Normalizing inputs puts observation components on comparable scales; normalizing critic outputs and targets consistently reduces sensitivity to reward scale. Continuous-action actors can output values in $[-1,1]$ and rescale them to the environment's action bounds.

4 Practical Implementation and Evaluation

To implement DQN using the Stable Baselines3 (SB3) library, key configuration points include:

- **Feature extractor:** `policy` selects the architecture. SB3's NatureCNN implements the original DQN's convolutional architecture; custom feature extractors are also supported.
- **Replay buffer size:** a large buffer (e.g., 1 million transitions) is recommended; this requires around 56 GBs of memory.
- **Learning starts:** SB3 delays learning until the buffer has been sufficiently filled.
- **Batch size:** number of samples drawn from the buffer per gradient step.
- **Soft/hard target update:** SB3 supports periodic hard updates or soft updates controlled by τ ($\tau = 1$ gives a hard update and is the default).
- **`target_update_interval`:** number of gradient steps between hard updates.
- **`train_freq` and `n_envs`:** train every `train_freq` environment steps, collecting each batch from all `n_envs` parallel environments.
- **Multi-step returns:** SB3 supports multi-step targets and trajectory sampling, which correspond to n-step Bellman updates.
- **Huber loss and clipping:** SB3 can apply Huber loss and clip the TD error when computing the loss.

Atari Benchmarks and Evaluation Metrics

Atari evaluation metrics compared DQN's scores to two baselines: a random agent and a human expert. The human-normalized score (HNS) was defined as $(\text{agent} - \text{random})/(\text{human} - \text{random})$. Comparisons can be unreliable because some games permit extremely large scores and random seeds can drastically affect results. Robust metrics like the inter-quartile mean (IQM) or median across many seeds mitigate these issues. The rliable package provides these metrics and bootstrap confidence intervals.

Benchmarks include the Atari 200M (million frames) or 1B (billion frames), where the number of steps is the same number but divided by 4 (4 frames are stacked). There are also papers for example giving the 1/3/5/10 most predictive games in order to get a more valid analysis without testing all games.

To make Atari environments less deterministic and to encourage robust policies, several forms of environment randomisation are used:

- **Noisy networks (NoisyNets):** learned noise in network weights makes Q-values stochastic, replacing ε-greedy exploration with parameter noise without randomising the environment itself.
- **Sticky actions:** probabilistically repeat the previous action despite a new request. Proposed in 2017 after DQN, this helps prevent exploitation of deterministic dynamics.
- **Random frame skipping:** instead of always repeating each action for a fixed number of frames (e.g., 4), choose a number of skipped frames randomly in a range.
- **Initial no-ops:** start each episode with a random number of no-op actions to randomise the initial state. A fixed number still gives a deterministic game.
- **Random action noise:** occasionally execute a random action instead of the policy's action. This is mostly discouraged, as it leads to high instability.

These techniques make the training environment more stochastic and reduce the risk of over-fitting to deterministic trajectories.